



4



7



1050 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

أهم التعليمات في MIPS

م. عبير الميا



RB Informatics; 09/09/2020

محتوى مجاني غير مخصص للبيع التجاري

بنيان الحاسوب ا

أولاً: تعليمات المقارنة في MIPS:

التعليمة الحقيقية (Set if less than): slt



ما معنى set؟ تعني شحن السجل ب 1 في حال تحقق الشرط وب 0 خلاف ذلك
ولهذه التعليمة 3 أشكال:

slt rd, rs, rt -> if (rs < rt) rd = 1 else rd = 0 - 1

slti rt, rs, immediate -> if (rs < immediate) rt=1 else rt= 0 - 2

sltu rd, rs, rt -> Set on less than unsigned - 3

ماذا تعني unsigned؟

لو كان العدد التالي بالتمثيل الثنائي: 11111111 هل هو سالب أم موجب؟
في حال تم التعامل معه كأنه unsigned فهو موجب وقيمه 255
أما لو تم التعامل بدون unsigned فهو سالب وتعرف قيمته بإيجاد المتمم الثنائي له وهو -1.

التعليمات الزائفة (مع التفرع):

1. تعليمة blt وما يكافئها من تعليمات حقيقية:

blt \$a0, \$a1, Label -> if \$a0 < \$a1 go to Label

else continue to the next line of code

slt \$at, \$a0, \$a1

bne \$at, \$zero, Label

(branch if greater than) Bgt

(branch if greater or equal) Bge

(branch if less than) Blt

(branch if less or equal) Ble

شرح التعليمات الحقيقية المكافئة ل `blt`:

أولا تخزين نتيجة المقارنة بين a_0 , a_1 في السجل at (المعالج يستخدم هذا السجل) ثم يقارن محتواه مع السجل الصفري فإذا لم يساوه بالقيمة فيعني أنه يحوي 1 فيتفرع إلى Label.

توضيح المقارنة مع السجل الصفري أسهل من شحن سجل ما بقيمة 1 والمقارنة معه.

2- تعليمة `ble` وما يكافئها من تعليمات حقيقية:

`ble $t0, $s0, Label -> if $t0 <= $s0 go to Label else continue to the next line of code`

`slt $at, $s0, $t0`

`beq $at, $zero, Label`

شرح التعليمات الحقيقية المكافئة ل `ble`:

نحن نريد التفرع إلى Label في حال $t_0 \leq s_0$ ولا يوجد سوى تعليمة `slt`، فكيف نتصرف؟

أولا : ما عكس $t_0 \leq s_0$ ؟ الجواب: $t_0 > s_0$ ، انطلاقا من هذه المعلومة يمكنني استخدام تعليمة `slt` للتحقق من أن السجل s_0 أصغر تماما من السجل t_0 ، ففي حال تحقق ذلك أنا أريد العكس أي $t_0 \leq s_0$ لذلك إذا كان محتوى السجل at يساوي الصفر يتفرع إلى Label أي أن محتوى السجل s_0 ليس أصغر من محتوى t_0 .

ثانيا: تعليمات الضرب والقسمة في MIPS:

1. تعليمة الضرب الحقيقية:

`mult register1, register2`

تقوم بضرب محتوى السجلين، ولكن أين ناتج عملية الضرب؟؟؟

إن كل سجل مؤلف من 32 بت فجداء سجلين يحتاج إلى 64 بت وهذا يعني تخزينه في سجلين أول سجل يسمى `low` يحتوي على ال 32 بت الدنيا من الناتج وسجل يسمى `high` يحتوي على 32 بت العليا من الناتج.

فمثلا:

`mult $s1, $s2`

`mfhi $t0`

هذه التعليمة تنقل ال 32 بت العليا من ناتج الضرب من السجل `hi` إلى السجل t_0 .

`mflo $t1`

هذه التعليمة تنقل ال 32 بت الدنيا من ناتج الضرب من السجل `lo` إلى السجل t_1 .

2. تعليمة الضرب الزائفة وما يكافئها:

التعليمة الزائفة:

mul \$t0, \$t1, \$t2 -> \$t0 = \$t1 x \$t2

mult \$t1, \$t2

mflo \$t0

وبما أنها تنقل فقط من السجل lo فقط بدون محتوى hi فيعني أننا يجب أن نكون متأكدين عند استخدام تعليمة الضرب الزائف أن ناتج عملية الضرب يكفيه 32 بت لتخزينه.

3. تعليمة القسمة الحقيقية:

div register , register2

الناتج أيضا يخزن في السجلين hi و lo، لكن ليس لأن الناتج كبير ولكن لأن القسمة لها ناتج ولها باقي. حيث يتم تخزين ناتج القسمة في lo وباقي القسمة في hi، مثال:

div \$s0, \$s1

mfhi \$t0 -> \$t0 = \$s0 % \$s1

mflo \$t1 -> \$t1 = \$s0 / \$s1

4. تعليمة القسمة الزائفة وما يكافئها:

التعليمة الزائفة:

div \$s0, \$t1, \$t2 -> \$s0 = \$t1 / \$t2

div \$t1, \$t2

mflo \$s0

ملاحظة: تعليمات الضرب والقسمة الحقيقيتان وتعليمتا mflo, mfhi هي من النمط R-Type



ثالثاً: تعليمات الإزاحة في MIPS:

للإزاحة نوعان: منطقية logical shift وحسابية arithmetic shift:

➤ الإزاحة المنطقية:

ولها نوعان:

إزاحة منطقية لليسار (sll (shift left logical): إزاحة لليسار وملء الخانات الفارغة بالأصفار.

إزاحة منطقية لليمين (srl (shift right logical): إزاحة لليمين وملء الخانات الفارغة بالأصفار.

ملاحظة: الإزاحة المنطقية اليسار تكافئ ضرب العدد ب(مقدار الإزاحة) 2^n ، والإزاحة المنطقية اليمين تكافئ تقسيم العدد على (مقدار الإزاحة) 2^n .

مثال:

ليكن العدد 4 المخزن في السجل \$t0\$ فيكون مخزن على الشكل التالي:

000000000000000000000000000000**00**100

وعند القيام ب:

```
sll $t1, $t0, 2
```

يُطَبِّحُ:

[illegible]

الشرح:

أولا شرح التعليلة:

```
sll $t1, $t0, 2
```

أي القيام ب left logical shift على محتوى السجل \$t0 بمقدار 2 وتخزين الناتج في السجل \$t1 .

ماذا يعنى إزاحة منطقية ليسار بمقدار 2؟؟

آخر بتين في العدد(بالأزرق) يحدفون ويضاف أصفار بمقدار الإزاحة في أول العدد(بالبنفسجي)، ويصبح العدد الناتج 16.

وعند القيام ب:

```
srl $t2, $t0, 1
```

يصبح: 000

أي تم حذف بتات بمقدار الإزاحة من اليمين وإضافة أصفار بمقدار الإزاحة في اليسار ويصبح الناتج 2.



Sometimes you win
Sometimes you learn

الإزاحة الحسابية:

وهي فقط إزاحة حسابية لليمين shift right arithmetic:

الفكرة هي: ننظر للبت اليسار للعدد قبل الإزاحة، نحذف بتات من يمين العدد بمقدار الإزاحة ثم نضيف نفس البت الأخير (اليساري) للعدد من اليسار قبل الحذف بمقدار الإزاحة والهدف من ذلك الحفاظ على إشارة العدد بعد الإزاحة

مثال:

```
sra $t1, $t0, 2 # $t0 = 1000100000000010000001110111111
```

أي قم بعمل sra لمحتوى السجل \$t0 وخرن الناتج في السجل \$t1.

فيصبح محتوى السجل \$t1:

```
# $t1 = 11100010000000001000000111011111
```

رابعا: تعليمات تخزين وتحميل بايت واحد في MIPS:

1. تعليمة lb(load byte):

تقوم بتحميل بايت واحد من موقع في الذاكرة إلى السجل.

```
lb register1, offset(register with memory address) -> lb $t1, 8($s0)
```

التعليمة السابقة تقوم بتحميل بايت من موقع الذاكرة المخزن في \$s0 بعد انزياح 8 بايتات وتخزنه في السجل \$t1.

2. تعليمة sb(store byte):

تقوم بحفظ بايت واحد من السجل (البتات الثمانية الدنيا) إلى الذاكرة.

```
sb register1, offset(register with memory address) -> sb $t1, 5($s0)
```

- ملاحظة 1: في تعليمتي lb, sb لا يشترط أن يكون ال offset من مضاعفات ال 4.

- ملاحظة 2: هناك تعليمة أخرى اسمها lbu(load bit unsigned) تقوم بنفس العمل الذي تقوم به lb، إذا ما الفرق؟

تعليمة lb تملأ باقي بتات السجل بنفس البت الأخير للبايت الذي تم تحميله، بينما تعليمة lbu تملأ باقي بتات السجل بأصفار.

- ملاحظة 3: يستخدم MARS نظام النهاية الصغرى Little Endian فعند تخزين كلمة في الذاكرة يضع البايت الأدنى في العنوان الأدنى، ولكن لو كان يستخدم نظام النهاية الكبرى Big Endian فكان سيخزن البايت الأعلى في العنوان الأدنى.

.data

مثال:

bt: .byte 'H','i','2','6'

sp: .space 4

w: .word 0x49345175

Data Segment							
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)
0x10010000	6 2 i H	1 \0 \0 2	I 4 Q u	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0
0x10010020	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0	\0 \0 \0 \0

تمارين:

1. بفرض محتوى \$t0 هو 0x12345678، ما هو ناتج العملية التالية 1، \$t0، \$sra \$t1، \$t0،

الحل:

نكتب العدد بالتمثيل الثنائي: 00010010001101000101011001111000

العملية هي shift right arithmetic أي نحذف بتات من اليمين بمقدار الإزاحة ونضيف بتات نفس البت الأخير للعدد بمقدار الإزاحة، فيصبح محتوى السجل \$t1 كالتالي:

0x091A2B3C أي بالسداسي عشر: 00001001000110100010101100111100

2. اكتب برنامجا بلغة MIPS يقوم بتشفير encryption لسلسلة نصية string منتهية بـ null بإجراء عملية XOR بين كل حرف ومفتاح التشفير ولتكن قيمته 0x2f وخن الناتج في مساحة فارغة محجوزة بالذاكرة crpt:

قبل الحل، معلومة سريعة: العدد XOR نفسه يساوي الصفر، إذا ال XOR عملية يمكن عكسها بسهولة. مثال: $A \text{ xor } B = C$ ، نقوم بعمل xor للطرفين مع B فنحصل على $A \text{ xor } B \text{ xor } B = C \text{ xor } B$ وبما أن $B \text{ xor } B = 0$ و أي عدد XOR 0 هو العدد نفسه تصبح المعادلة: $A = C \text{ xor } B$. لذلك استخدمنا ال xor في التشفير، يكفي معرفة مفتاح التشفير لنقوم بفك الشيفرة



الحل أسفل عكازتي

```

.data:
string: .ascii "Hello"
crpt: .space 6
key: .byte 0x2f
.text:
#loading addresses from memory
la $t0, string
la $t1, crpt
la $t4, key
# since encryption key is 2 hexa digits, meaning 8 bits = 1 byte, we used lb
# to load the key from the address
lb $t2, 0($t4)
# loop to do an XOR between each char of the string and the Encryption Key

# we used the $t4 register to store the address of the key
# since we have the value of the key, we don't need the address anymore
# so we are using the $t4 register to temporarily store the value of the operation ([char from array] xor key)

Encrypt_Loop:
    lb $t3, 0($t0) #load the first byte(char is 1 byte) to $t3
    # compare this char with zero
    # since it's an ascii, the string must end with '\0' which equals 0
    # so the loop breaks
    beq $t3, $zero, End
    # if the char is not 0, the code continues sequentially
    # we xor this char with the key
    xor $t4, $t3, $t2
    # we store the encrypted char inside the reserved space crpt with offset 0
    sb $t4, 0($t1)
    # increment the string address by one, so now it points on the following char
    addi $t0, $t0, 1
    # increment the crpt address by one, so now it points on the following empty space
    addi $t1, $t1, 1
    j Encrypt_Loop

End:

```

ملاحظة: للقيام بفك تشفير النص السابق نقوم بعمل xor لكل حرف من النص المشفر مع نفس مفتاح التشفير فنحصل على الرسالة الأصلية.

3. بفرض محتوى السجل: \$t0 هو 0xFFAA4422 أوجد ناتج تنفيذ التعليمات التالية:

sll \$t1, \$t0, 8 # shift left logical \$t0 by 8 bits -> \$t1

srl \$t2, \$t1, 16 # shift right logical by 16 bits -> \$t2

الحل:

دوما نأخذ العدد بتمثيله الثنائي (سواء كان معطى بالنظام العشري أو السداسي عشري):
 إن العدد 0xFAA4422 هو 1111111101010100100010000100010، فبعد تنفيذ أول تعليمة يصبح:

$$\$t1 = 10101010010001000010001000000000$$

حيث تم حذف 8 بتات من اليسار وإضافة 8 أصفار على اليمين، وأصبح العدد: 0xAA442200
 وبعد تنفيذ التعليمة الثانية يصبح:

$$\$t2 = 000000000000000001010101001000100$$

حيث تم حذف 16 بت من اليمين وإضافة 16 صفر على اليسار وأصبح العدد: 0x0000AA44، أي 0xAA44

ملاحظة: تم عزل الباي 1 و 2 (الباي = 8 بتات = خانتين من التمثيل الست عشري) (نبدأ العد من الصفر ومن الأدنى) من السجل \$t0 بعد التعليمات ووضعهم في الباي 0 و 1 في السجل \$t2.

تمرين: قم بعزل الباي 2 من السجل \$t0 السابق وضعه في الباي 0

```
1 li $t0, 0xffaa4422
2
3 sll $t1, $t0, 8
4 srl $t2, $t1, 24
```

→

\$t0	8	0xffaa4422
\$t1	9	0x22000000
\$t2	10	0x00000022



التمرين الأول:

بفرض أن السجل \$t0 يحتوي على القيمة الست عشرية المكونة من 32 بت 0x89ABCDEF , اكتب تعليمات MIPS باستخدام الإزاحة المنطقية فقط لاستخراج البايت الثاني من الأعلى (0xAB) ووضعه في موضع البايت الأدنى، بحيث تصبح القيمة النهائية 0x000000AB.

الجواب:

```
sll $t0 , $t0 , 8
srl $t0 , $t0 , 24
```

التمرين الثاني:

نص السؤال : بفرض أن السجل \$s0 يحتوي على القيمة 15 والسجل \$s1 يحتوي على القيمة 5. اكتب مقطعاً برمجياً بلغة MIPS باستخدام تعليمات الضرب الحقيقية لضربهما واسترجاع الناتج (32 بت) إلى السجل \$t0. بعد ذلك، استخدم تعليمات المقارنة الحقيقية للتحقق مما إذا كان الناتج أصغر تماماً من 100. إذا كان الشرط محقق، تفرع إلى Label يُدعى IsLess.

الجواب:

```
mult $s0 , $s1
mflo $t0

addi $s2 , $zero , 100
slt $v0 , $t0 , $s2
bne $v0 , $0 , IsLess
IsLess:
```

التمرين الثالث:

السؤال : اكتب حلقة تكرارية بلغة التجميع MIPS تقوم بقراءة سلسلة نصية منتهية بمحرف فارغ مخزنة في العنوان الموجود في السجل \$t0، بايئاً تلو الآخر. إذا كان البايت يساوي قيمة ASCII للحرف 'A' (قيمه بال ascii هي (0x41))، فاستبدله في الذاكرة بالحرف 'Z' (قيمه بال ascii هي (0x5A)). يجب أن تتوقف الحلقة عندما تصادف المحرف الفارغ (0x00).

```
.data
string: .asciiz "abcAefAhiakl"
.text
la $t0 , string
addi $a0 , $zero , 0x41 #the value of A in the ascii table
addi $a1 , $zero , 0x5A #the value of Z in the ascii table
```

```

loop:
lb $v0 , 0($t0) #load the value of the byte at the current
address
beq $v0 , $0 , terminate #if it's equal to zero terminate the
loop
bne $v0 , $a0 , dont
#if the loaded byte equals "A" then replace it with "Z"
sb $a1 , 0($t0)
dont:
#if not then continue normally
addi $t0 , $t0 , 1
j loop
terminate:

```

أسئلة نظرية: السؤال الأول: تعليمة `slt $t0, $s1, $s2` تفعله التعليمة `slt` ما الذي تفعله التعليمة `slt` فعلياً؟

(أ) تطرح `$s2` من `$s1` وتُخزن الناتج في `$t0`.

(ب) تضع القيمة 1 في السجل `$t0` إذا كان `$s1` أصغر تماماً من `$s2`؛ وإلا فإنها تضع القيمة 0.

(ج) تتفرع إلى العنوان الموجود في `$t0` إذا كان `$s1` أصغر من `$s2`.

(د) تزيح بتات `$s1` إلى اليسار بمقدار القيمة المحددة في `$s2`.

الإجابة الصحيحة:

(ب) تضع القيمة 1 في السجل `$t0` إذا كان `$s1` أصغر تماماً من `$s2`؛ وإلا فإنها تضع القيمة 0.

السؤال الثاني: الضرب والسجلات الخاصة عند استخدام تعليمة الضرب الحقيقية `mult $s1, $s2`، أين يذهب الناتج المكون من 64 بت؟

(أ) يتم ضغط الناتج بالكامل في `$s1`، مما يسبب طفحاً (Overflow) إذا كان كبيراً جداً.

(ب) يُخزن الناتج في عنوان الذاكرة الذي يُؤشر عليه مؤشر المكسدس (Stack Pointer).

(ج) تذهب الـ 32 بت العليا إلى السجل `hi`، والـ 32 بت الدنيا إلى السجل `lo`.

(د) تذهب الـ 32 بت الدنيا إلى `$s1`، والـ 32 بت العليا إلى `$s2`.

الإجابة الصحيحة:

(ج) تذهب الـ 32 بت العليا إلى السجل `hi`، والـ 32 بت الدنيا إلى السجل `lo`.

السؤال الثالث: الإزاحة لليمين (المنطقية مقابل الحسابية) ما هو الفرق الأساسي بين الإزاحة المنطقية لليمين (`srl`) والإزاحة الحسابية لليمين (`sra`) ؟

- (أ) تملأ srl الخانات الفارغة اليسرى بالآحاد، بينما تملأها sra بالأصفر.
- (ب) تُستخدم srl لعمليات الضرب، بينما تُستخدم sra لعمليات القسمة.
- (ج) تنظر sra إلى البت الأكثر أهمية (بت الإشارة) وتنسخه للحفاظ على إشارة العدد، بينما تملأ srl الخانات اليسرى دائماً بالأصفر.
- (د) لا يوجد فرق وظيفي؛ هما مجرد اسمان مختلفان لنفس التعليمة حسب المترجم.

الإجابة الصحيحة:

(ج) تنظر sra إلى البت الأكثر أهمية (بت الإشارة) وتنسخه للحفاظ على إشارة العدد، بينما تملأ srl الخانات اليسرى دائماً بالأصفر.

السؤال الرابع: تحميل بايت 1b مقابل 1bu إذا قمت بتحميل بايت من الذاكرة قيمته الثنائية (0xFF) (11111111)، ماذا سيحدث إذا استخدمت تعليمة 1b مقارنة بتعليمة 1bu؟

- (أ) تملأ 1b الـ 24 بت المتبقية من السجل بالآحاد (تمديد الإشارة)، بينما تملأها 1bu بالأصفر (تمديد بالأصفر).
- (ب) تملأ 1bu الـ 24 بت المتبقية بالآحاد، بينما تملأها 1b بالأصفر.
- (ج) كلتا التعليمتين تفعّلان نفس الشيء تماماً؛ حرف "u" يعني التحديث (Update) فقط.
- (د) تُحمّل 1b البايت في السجل 1o، بينما تُحمّل 1bu في سجل أغراض عامة عادي.

الإجابة الصحيحة:

(أ) تملأ 1b الـ 24 بت المتبقية من السجل بالآحاد (تمديد الإشارة)، بينما تملأها 1bu بالأصفر (تمديد بالأصفر).

السؤال الخامس: توسيع التعليمات الزائفة كيف يتم تحويل التعليمة الزائفة `ble $t0, $s0, Label` (تفرع إذا كان أصغر أو يساوي) عادةً بواسطة المُجمّع (Assembler) إلى تعليمات حقيقية؟

- (أ) `sub $at, $t0, $s0` (متبوعة بـ `Label` و `$zero` و `$at`) (`beq $at, $zero, Label`)
- (ب) `slt $at, $s0, $t0` (متبوعة بـ `Label` و `$zero` و `$at`) (`beq $at, $zero, Label`)
- (ج) `slt $at, $t0, $s0` (متبوعة بـ `Label` و `$zero` و `$at`) (`bne $at, $zero, Label`)
- (د) `sltu $at, $t0, $s0` (متبوعة بـ `Label` و `$zero` و `$at`) (`bge $at, $zero, Label`)

الإجابة الصحيحة:

(ب) `slt $at, $s0, $t0` (متبوعة بـ `Label` و `$zero` و `$at`) (`beq $at, $zero, Label`)